Algorithmique 3 : structures de données arborescentes

Projet

Génération de textes aléatoires

Généralités

Le projet consiste à écrire un programme en C permettant, à partir d'un texte donné, d'en générer un autre qui, aléatoire, est basé sur des propriétés statistiques du premier. Pour fixer les idées, voici un extrait de *Knock* de Jules Romains, où les symboles ¶ et ↩ signifient respectivement la fin d'une ligne de texte et la continuation d'une ligne de texte sans coupure de mot :

```
Knock: De quoi souffrez-vous?¶
Le Tambour: Attendez que je réfléchisse! (Il rit.) Voilà. Quand j'ai dîné, il↩
 y a des fois que je sens une espèce de démangeaison ici. (Il montre le haut↩
 de son épigastre.) Ça me chatouille, ou plutôt, ça me grattouille.¶
Knock, d'un air de profonde concentration: Attention. Ne confondons pas.↩
 Est-ce que ça vous chatouille, ou est-ce que ça vous grattouille?¶
Le Tambour: Ça me grattouille. (Il médite.) Mais ça me chatouille bien un peu↩
 aussi.¶
Knock: Désignez-moi exactement l'endroit.¶
Le Tambour: Par ici.¶
Knock: Par ici... où cela, par ici?¶
Le Tambour: Là. Ou peut-être là... Entre les deux.¶
Knock: Juste entre les deux? ... Est-ce que ça ne serait pas plutôt un rien à↩
 gauche, là, où je mets mon doigt?¶
Le Tambour: Il me semble bien.¶
Knock: Ça vous fait mal quand j'enfonce le doigt?¶
Le Tambour: Oui, on dirait que ça me fait mal.¶
Knock: Ah! ah! (Il médite d'un air sombre.) Est-ce que ça ne vous grattouille↩
 pas davantage quand vous avez mangé de la tête de veau à la vinaigrette?¶
Le Tambour: Je n'en mange jamais. Mais il me semble que si j'en mangeais,↩
 effectivement, ça me grattouillerait plus.¶
```

Et voici l'un des textes aléatoires qui a été généré à partir du précédent :

```
Knock: De quoi souffrez-vous? Le Tambour: Là. Ou peut-être là... Entre les   ↩
 deux. Knock: Juste entre les deux? ... Est-ce que ça vous chatouille, ou    ↩
 est-ce que ça ne vous grattouille pas davantage quand vous avez mangé de la ↩
 tête de veau à la vinaigrette? Le Tambour: Là. Ou peut-être là... Entre les ↩
 deux. Knock: Juste entre les deux? ... Est-ce que ça ne serait pas plutôt un↩
 rien à gauche, là, où je mets mon doigt? Le Tambour: Oui, on dirait que ça  ↩
 vous grattouille? Le Tambour: Là. Ou peut-être là... Entre les deux. Knock: ↩
 Juste entre les deux? ... Est-ce que ça vous grattouille? Le Tambour: Par   ↩
 ici. Knock: Par ici... où cela, par ici? Le Tambour: Ça me grattouille.     ↩
 Knock, d'un air de profonde concentration: Attention. Ne confondons pas.    ↩
 Est-ce que ça vous chatouille, ou est-ce que ça me fait mal. Knock: Ah! ah! ↩
 (Il médite d'un air de profonde concentration: Attention. Ne confondons pas.¶
```

Description de l'algorithme

L'algorithme de génération aléatoire d'un texte se subdivise en deux phases : une phase de lecture du texte source et, conjointement, de construction de la structure qui permettra la génération ; la phase de génération du texte cible à proprement parler.

En plus du texte source, l'algorithme possède une autre donnée : un naturel appelé *ordre (de la génération)* et noté n dans la suite.

Durant la première phase, l'algorithme construit une table qui à des n-uplets de mots, les clés de la table, associe une liste de mots, les valeurs de la table. À la terminaison de cette phase, les clés de la table correspondent aux suites de n mots consécutifs figurant dans le texte source et la valeur associée à toute clé donnée correspond à la liste des mots qui suivent immédiatement les n mots formant la clé.

Dans l'illustration donnée plus haut, l'ordre n vaut 2. Si l'on considère qu'un mot correspond à une suite maximale de caractères qui ne sont pas des caractères d'espacement, la table construite contient 174 clés. Voici des extraits de cette table où, pour aider à les identifier, les mots sont écrits entre crochets :

2-uplets (clés)	*liste des successeurs (valeurs)*
([Knock:], [De])	⟨[quoi]⟩
([De], [quoi])	⟨[souffrez-vous?]⟩
([quoi], [souffrez-vous?])	⟨[Le]⟩
([souffrez-vous?], [Le])	⟨[Tambour:]⟩
([Le], [Tambour:])	⟨[Attendez], [Ça], [Par], [Là.], [Il], [Oui,], [Je]⟩
([Tambour:], [Attendez])	⟨[que]⟩
([Attendez], [que])	⟨[je]⟩
([que], [je])	⟨[réfléchisse!], [sens]⟩
([je], [réfléchisse!])	⟨[(Il]⟩
([réfléchisse!], [(Il])	⟨[rit.)]⟩
([(Il], [rit.)])	⟨[Voilà.]⟩
([rit.)], [Voilà.])	⟨[Quand]⟩
([Voilà.], [Quand])	⟨[j'ai]⟩
([Quand], [j'ai])	⟨[dîné,]⟩
([j'ai], [dîné,])	⟨[il]⟩
([dîné,], [il])	⟨[y]⟩
([il], [y])	⟨[a]⟩
([y], [a])	⟨[des]⟩
([a], [des])	⟨[fois]⟩
([des], [fois])	⟨[que]⟩
([fois], [que])	⟨[je]⟩
…	
([Ça], [me])	⟨[chatouille,], [grattouille.]⟩
([me], [chatouille,])	⟨[ou]⟩
([chatouille,], [ou])	⟨[plutôt,], [est-ce]⟩
([ou], [plutôt,])	⟨[ça]⟩
([plutôt,], [ça])	⟨[me]⟩
([ça], [me])	⟨[grattouille.], [chatouille], [fait], [grattouillerait]⟩
…	
([Attention.], [Ne])	⟨[confondons]⟩

2-uplets (clés)	*liste des successeurs (valeurs)*
([Ne], [confondons])	⟨[pas.]⟩
([confondons], [pas.])	⟨[Est-ce]⟩
([pas.], [Est-ce])	⟨[que]⟩
([Est-ce], [que])	⟨[ça], [ça], [ça]⟩
([que], [ça])	⟨[vous], [vous], [ne], [me], [ne]⟩
([ça], [vous])	⟨[chatouille,], [grattouille?]⟩
([vous], [chatouille,])	⟨[ou]⟩
([ou], [est-ce])	⟨[que]⟩
([est-ce], [que])	⟨[ça]⟩
([vous], [grattouille?])	⟨[Le]⟩
([grattouille?], [Le])	⟨[Tambour:]⟩
([Tambour:], [Ça])	⟨[me]⟩
…	
([Mais], [il])	⟨[me]⟩
([il], [me])	⟨[semble]⟩
([semble], [que])	⟨[si]⟩
([que], [si])	⟨[j'en]⟩
([si], [j'en])	⟨[mangeais,]⟩
([j'en], [mangeais,])	⟨[effectivement,]⟩
([mangeais,], [effectivement,])	⟨[ça]⟩
([effectivement,], [ça])	⟨[me]⟩
([me], [grattouillerait])	⟨[plus.]⟩
([grattouillerait], [plus.])	⟨⟩

Observez sur ce simple extrait que :

— plusieurs mots identiques peuvent figurer dans une même liste de successeurs, l'ajout à ces listes étant inconditionnel et pas ensembliste. D'où les occurrences multiples de [ça] dans la liste de la clé ([Est-ce], [que]), de [vous] et de [ne] dans celle de la clé ([que], [ça]) ;

— les clés d'une part et les mots qui figurent dans les listes de successeurs d'autre part sont rangés selon leur ordre d'apparition dans le texte source. Cela facilite la compréhension mais n'a aucune importance vu l'utilisation qui doit en être faite.

Première mouture de l'algorithme

Voici la formulation de la phase de l'algorithme consacrée à la lecture et à la construction de la table :

initialiser la table au vide
lire les n premiers mots du texte source et les affecter à $u_0, u_1, \ldots, u_{n-1}$
initialiser la liste des mots associés à la clé $(u_0, u_1, \ldots, u_{n-1})$ au vide dans la table
tant que au moins un mot reste à lire dans le texte source **faire**
    lire le premier des mots restant à lire et l'affecter à v
    ajouter v à la liste des mots associés à la clé $(u_0, u_1, \ldots, u_{n-1})$ dans la table
    remplacer $u_0, u_1, \ldots, u_{n-1}$ par $u_1, \ldots, u_{n-1}, v$
    si la clé $u_0, u_1, \ldots, u_{n-1}$ n'est pas déjà dans la table **alors**
        initialiser la liste des mots associés à la clé $(u_0, u_1, \ldots, u_{n-1})$ au vide dans la table

Passons maintenant à la seconde phase de l'algorithme, la phase de génération aléatoire. L'idée générale est la suivante : étant donné une clé $(u_0, u_1, \ldots, u_{n-1})$ de la table, il suffit, pour se conformer aux propriétés statistiques du texte source, de sélectionner aléatoirement l'un des

successeurs dans la liste associée à cette clé ; ce mot, disons v, est alors produit, puis utilisé pour former la clé suivante, $(u_1, \ldots, u_{n-1}, v)$, qui servira à son tour pour générer les prochains mots. La génération débute avec la clé formée par les n premiers mots du texte source.

Voici en conséquence la formulation de la seconde phase :

affecter à $u_0, u_1, \ldots, u_{n-1}$ les n premiers mots du texte source
produire $u_0, u_1, \ldots, u_{n-1}$ dans cet ordre
tant que la production peut se poursuive **faire**
    choisir aléatoirement un mot dans la liste associée à la clé $(u_0, u_1, \ldots, u_{n-1})$ dans la table
        et l'affecter à v
    produire v
    remplacer $u_0, u_1, \ldots, u_{n-1}$ par $u_1, \ldots, u_{n-1}, v$

Il reste à approfondir la condition d'itération de la boucle, « la production peut se poursuivre ». La génération telle qu'elle vient d'être décrite peut en effet connaitre deux écueils.

Premier écueil : boucler possiblement indéfiniment. Comme dans notre exemple, et toujours à l'ordre $n = 2$, avec, pour faire court :

```
Knock: De quoi souffrez-vous? Le Tambour: Attendez que je réfléchisse! (Il↩
 rit.) Voilà. Quand j'ai dîné, il y a des fois que je réfléchisse! (Il rit.)↩
 Voilà. Quand j'ai dîné, il y a des fois que je réfléchisse! (Il rit.) Voilà.↩
 Quand j'ai dîné, il y a des fois que je réfléchisse! (Il rit.) Voilà. Quand↩
 j'ai dîné, il y a des fois que je réfléchisse! (Il rit.) Voilà. Quand j'ai
```

Deuxième écueil : planter parce que la liste associée à la clé $(u_0, u_1, \ldots, u_{n-1})$ est vide. Comme dans notre exemple, après avoir produit

```
Knock: De quoi souffrez-vous? Le Tambour: Ça me chatouille, ou plutôt, ça me↩
 grattouillerait plus.
```

Le remède est simple. Il suffit de disposer du nombre maximum de mots à produire, disons m, lequel peut être une constante ou une donnée supplémentaire. Puis, dans la condition d'itération de la boucle, de vérifier si le nombre de mots déjà produits est strictement inférieur à m et si liste associée à la clé $(u_0, u_1, \ldots, u_{n-1})$ n'est pas vide.

Simplification par le mot vide

La formulation des deux phases précédentes donnée ci-dessus, même améliorée avec la remédiation qui vient d'être proposée, n'est toutefois toujours pas satisfaisante, et pour deux raisons.

Première raison : une propriété du texte source n'a pas été retenue. Si la suite des n derniers mots du texte source y apparait p fois, la clé associée à ces n mots doit proposer p successeurs, dont l'un signifie l'arrêt de la production. Ce n'est pas le cas pour notre exemple de référence. Mais cela l'aurait été s'il avait été tronqué immédiatement après le « `Ça me grattouille.` » de la deuxième intervention du Tambour : la clé ([`me`], [`grattouille.`]) aurait eu pour successeurs « `Knock,` » et « plus rien ».

Deuxième raison : il n'est pas tenu compte du fait que le nombre de mots figurant dans le texte source pourrait être strictement inférieur à n.

La prise en compte d'un mot qui ne peut pas être un mot du texte source au sens retenu ici, à savoir le mot vide ε, permet de solutionner les deux problèmes à la fois. Il faut toutefois accepter d'introduire dans la table jusqu'à n clés supplémentaires et jusqu'à $n+1$ mots supplémentaires dans les listes des successeurs ; ce qui n'est pas excessif. Il s'agit :

— concernant les au plus n premiers mots t_0, t_1… du texte source, des couples clé-liste de la forme $((\varepsilon, \ldots, \varepsilon, t_0, t_1, \ldots, t_{k-1}), \langle t_k \rangle)$ pour $0 \leqslant k < \min\{n, d\}$, où d est le nombre de mots du texte source ;

— concernant les au plus n derniers mots $\ldots t_{d-2}$, t_{d-1} du texte source, du couple clé-liste $((t_{d-n}, t_{d-n+1}, \ldots, t_{d-1}), \langle\varepsilon\rangle)$ en posant $t_k = \varepsilon$ pour $k < 0$.

Avec cette solution, il y a $174 + 2 = 176$ clés pour notre exemple de référence et une liste modifiée :

2-uplets (clés)	*liste des successeurs (valeurs)*
$([], [])$	$\langle$[`Knock:`]$\rangle$
$([], [\texttt{Knock:}])$	$\langle$[`De`]$\rangle$
$([\texttt{grattouillerait}], [\texttt{plus.}])$	$\langle[]\rangle$

le mot vide étant ici noté [].

Deuxième mouture de l'algorithme

L'algorithme complet peut dès lors s'exprimer comme suit :

♯ *phase de lecture et de construction de la table*
initialiser la table au vide
affecter ε à $u_0, u_1, \ldots, u_{n-1}$
faire
    si la clé $u_0, u_1, \ldots, u_{n-1}$ n'est pas déjà dans la table **alors**
        initialiser la liste des mots associés à la clé $(u_0, u_1, \ldots, u_{n-1})$ au vide dans la table
    si il ne reste aucun mot à lire dans le texte source **alors**
        affecter ε à v
    sinon
        lire le premier des mots restant à lire et l'affecter à v
    ajouter v à la liste des mots associés à la clé $(u_0, u_1, \ldots, u_{n-1})$ dans la table
    remplacer $u_0, u_1, \ldots, u_{n-1}$ par $u_1, \ldots, u_{n-1}, v$
tant que $v \neq \varepsilon$
♯ *phase de génération aléatoire*
affecter ε à $u_0, u_1, \ldots, u_{n-1}$
choisir aléatoirement un mot dans la liste associée à la clé $(u_0, u_1, \ldots, u_{n-1})$ dans la table et
        l'affecter à v
tant que $v \neq \varepsilon \wedge$ le nombre de mots produits est strictement inférieur à m **faire**
    produire v
    remplacer $u_0, u_1, \ldots, u_{n-1}$ par $u_1, \ldots, u_{n-1}, v$
    choisir aléatoirement un mot dans la liste associée à la clé $(u_0, u_1, \ldots, u_{n-1})$ dans la table
            et l'affecter à v

Mise en œuvre

L'exécutable doit être nommé `randtext`.
Sa syntaxe est

```
randtext [OPTION...] [FILE]
```

Si aucun fichier ne figure sur la ligne de commande, le texte source est lu sur l'entrée standard.

Le programme, écrit en C, doit pouvoir être compilé avec `gcc` branche 13.3 (minimum) et supporter les options de compilation usuelles : `-std=c2x`, `-Wall`, `-Wconversion`, `-Werror`, `-Wextra`, `-Wpedantic`, `-Wwrite-strings` et `-O2`.

Les sources C doivent être mis en forme par `uncrustify` selon le fichier de configuration en usage dans les salles de TP. La longueur d'aucune de leurs lignes ne doit excéder 80.

Les types, sous-programmes et paramètres doivent être correctement nommés. Toute duplication de code est à éviter. En particulier, les éventuels caractères, chaines et nombres magiques doivent être nommés.

L'utilisation de variables globales doit être évitée.

À la base du développement doit être utilisée au moins l'un des deux en-têtes `"hashtable.h"` ou `"bst.h"` qui figurent dans l'archive des sources mise à disposition sur UniversiTICE. Pour rappel, il est interdit d'y apporter la moindre modification.

La partie implantation associée à l'en-tête `"bst.h"` se doit de traiter les arbres binaires de recherche comme des AVL.

Des en-têtes, sources et modules additionnels peuvent être utilisés ou développés, par exemple pour gérer les options, la récupération de mots dans un flot texte…

Le module `malloccp` doit être intégré à toutes les parties implantations des modules ainsi qu'au source qui contient la fonction principale `main`. Le fichier `makefile` doit en tenir compte. Pour qu'une batterie de tests automatiques soient facilement réalisables, il est imposé que la première ligne du fichier `makefile` soit « `MALLOCCP = 0` » et que le texte « `-DMALLOCCP=$(MALLOCCP)` » ait été ajouté à la variable `CFLAGS`. Ni la partie en-tête ni la partie implantation du module `malloccp` ne doivent être modifiées.

Toutes les zones mémoires explicitement allouées doivent être désallouées avant la fin de l'exécution, même en cas d'erreur. Jamais deux chaines de caractères de même valeur ne doivent être allouées.

Tout message d'avertissement ou d'erreur doit être envoyé vers la sortie erreur. Si une erreur de syntaxe est détectée sur la ligne de commande, le code d'erreur qui doit être renvoyé par la fonction standard `exit` doit être 64. Pour tout autre type d'erreur, il convient de renvoyer `EXIT_FAILURE`.

Par défaut, un mot correspond à une suite maximale de caractères qui ne sont pas des caractères d'espacement.

Le texte source est lu sur l'entrée standard ou dans un fichier. Le texte produit est envoyé sur la sortie standard. Pour le texte produit : par défaut, deux mots consécutifs sont séparés par une (et une seule) espace ; le premier mot n'est précédé d'aucun caractère ; le dernier mot est immédiatement suivi du caractère fin de ligne (et de lui seul).

Des mots lus, seuls les préfixes d'une taille maximale fixée sont retenus. Les octets qui excèdent la limite ne doivent en aucun cas retenus ; un message d'avertissement approprié doit être émis pour chaque débordement. Par défaut, la limite est fixée à 63.

La fonction standard `rand` doit être utilisée pour guider la génération des mots. Par défaut, le germe du générateur pseudo-aléatoire doit être fixé à 1 au début l'exécution.

Par défaut :
— l'ordre n de la génération est fixé à 1 ;
— la limite m du nombre de mots (non vides) à produire est fixée à 100.

Quatre options doivent obligatoirement être supportées :

`-s NUMBER` ou `--seed=NUMBER` : fixer à (la valeur) `NUMBER` le germe du générateur pseudo-aléatoire ;

`-t` ou `--table` : supprimer l'affichage normal ; le remplacer par l'affichage de la table. Pour chaque clé, sur une même ligne, sont produits les mots qui la constitue puis les mots de la liste des successeurs. Tous ces mots sont séparés par le caractère tabulation (et par ce seul caractère). Le premier mot n'est précédé d'aucun caractère. Le dernier n'est suivi d'aucun caractère (autre que le caractère de fin de ligne de la ligne) ;

`-w` ou `--words` : supprimer l'affichage normal ; le remplacer par l'affichage des mots différents qui apparaissent dans le texte source. Un mot est affiché par ligne ; **MÀJ** 14-05 ⊕différents⊕ (Maram)

`-U` ou `--usage` : afficher un court mode d'emploi qui détaille la syntaxe en montrant les options disponibles (et elles seules).

D'autres options peuvent être développées :

`-l NUMBER` ou `--limit=NUMBER` : fixer la limite m du nombre de mots à produire à `NUMBER`. La limite 0 doit être supportée ;

`-o NUMBER` ou `--order=NUMBER` : fixer l'ordre n de la génération à `NUMBER`. Il est entendu que la valeur de `NUMBER` ne peut être nulle. Elle peut être limitée, par exemple à 1, 2, 5, 10 ou encore 100 ;

`-S NUMBER` ou `--size=NUMBER` : fixer la taille maximale des préfixes à `NUMBER`. Il est entendu que la valeur de `NUMBER` ne peut être nulle. Elle peut être limitée, par exemple à 31, 63 ou encore 127.

Des options qui permettent de modifier la mise en forme des productions peuvent également être développées :

`-L` ou `--keep-end-of-lines` : garder lors de la lecture des mots les éventuels caractères de fin de ligne qui les suivent immédiatement. Les caractères supplémentaires s'entendent dans la limite précisée pour la taille maximale des préfixes. Lors de l'affichage de la table ou de la liste des mots, les mots suivis des caractères de fin de ligne doivent être écrits suivis d'une (seule) espace puis d'autant de caractères « `L` » que de caractères de fin de ligne ;

`-W NUMBER` ou `--width=NUMBER` : fixer à `NUMBER` la largeur d'affichage en complétant si nécessaire avec des espaces « à droite » et une espace « à gauche » . Dans le cas d'une largeur non nulle, aucun mot de longueur inférieure ou égale à `NUMBER` ne doit pas apparaitre comme coupé si l'affichage a lieu dans une fenêtre de largeur `NUMBER`. La largeur 0 signifie qu'il n'y a aucune limitation à la largeur. La largeur peut être limitée, par exemple à 80 ou 255. La valeur par défaut est 0.

Le développement de l'usuelle option d'affichage de l'aide `-?` ou `--help` est souhaitée mais n'est pas obligatoire. Elle doit en tous cas être en conformité avec l'exigence émise pour l'option `-U` ou `--usage`.

Sans obligation non plus, et à l'instar de certaines commandes Linux, « `-` » et « `--` » peuvent être donnés seuls sur la ligne de commande. « `-` » est utilisé à la place d'un nom de fichier pour signifier explicitement la lecture sur l'entrée standard. « `--` » signifie la fin des options : après ne doivent se trouver que des opérandes (autrement dit des non-options).

Tous les paramètres figurant sur la ligne de commande doivent être pris en compte. En particulier, l'exécutable doit pourvoir différencier parmi les paramètres ceux qui relèvent de la catégorie option et ceux qui n'en relèvent pas. Face à une option qu'il ne supporte pas, l'exécutable ne doit pas planter mais prévoir une fin d'exécution normale avec un message d'erreur approprié.

Il est possible de ne développer que des options courtes ou bien que des options longues.

Modalités

Le projet est facultatif : à l'éventuelle absence au projet peut être substituée la note de l'examen de TP.

Le projet ne s'adresse qu'aux étudiant·es qui se rendent aux séances de TP et qui travaillent régulièrement lors de ces séances.

Vous pouvez réaliser le projet seul·e ou en binôme. Vous êtes incité·e à le réaliser en binôme.
La réalisation du projet comprend une soutenance orale individuelle.

Votre projet doit être compilable et exécutable sur les machines des salles de travaux pratiques du département d'informatique et sous Mint.

Il doit répondre en tous points aux exigences signifiées sur ce support, réponses à la foire aux questions comprises.

Il sera en partie testé avec des outils automatiques.

Votre projet doit être rendu avec tous les fichiers sources nécessaires, un fichier `makefile` pour produire l'exécutable (`make` sans paramètre ou avec les paramètres `all` ou `randtext`) et réaliser le nettoyage d'après production (`make` avec le paramètre `clean`), un rapport succin de développement et un deuxième fichier `makefile` pour confectionner l'archive compressée au format `.tar.gz` (`make` sans paramètre ou avec le paramètre `dist`). Le nom complet de l'archive doit être *identifiant*`_projet.tar.gz`, où *identifiant* est votre identifiant de 8 caractères (*login*) ou celui de l'autre membre du binôme.

À la racine de l'archive : les dossiers des modules, le dossier `randtext/`, le dossier `doc/`, le fichier `makefile` de confection de l'archive. Dans le dossier `randtext/` : le source contenant la fonction principale `main` et le fichier `makefile` de production de l'exécutable et de nettoyage. Dans le dossier `doc/`, le rapport.

Le rapport doit être fourni sous forme électronique au format `pdf`. Il se doit de préciser l'objet de chacun des modules, de décrire l'implantation (dessins souhaités), de commenter des exemples, d'expliciter les éventuelles limitations. Il doit être nommé `randtext.pdf`.

Aucun autre fichier, dossier ou archive que celles qui viennent d'être précisées ne doit être joint. En particulier, aucun des textes ayant servi aux exemples.

MÀJ 21-05
Précision de la date.

Vous remettrez votre projet au plus tard le jeudi 28 mai 2026 à 12 h 34.

Hors jeux

MÀJ 21-05
Ajout du corps de la section.

La note attribuée au projet est 0/20 en cas :

— de demande expresse de la part de l'étudiant·e,

— d'absence de dépôt,

— d'erreur d'identifiant dans le nom de l'archive,

— d'archive non conforme : format, contenu (fichier manquant, tel un source, le fichier `makefile` ou le rapport ; fichier en trop, tel un exécutable, un fichier objet, un fichier temporaire, un fichier texte qui n'est ni un source ni un fichier `makefile`),

— d'erreur apparaissant à la compilation,

— de non-respect de la mise en forme standard,

— de recopie dans un source quelconque d'une structure normalement cachée dans la partie implantation d'un module,

— hors interface `"holdall.h"`, d'utilisation d'une interface dont les types élémentaires « à la base du développement » ne sont pas `const void *`,

— d'utilisation d'un forceur de type pour contrecarrer l'aspect non mutable d'une donnée (sauf pour se conformer au type de retour d'une fonction et, dans ce cas, avec une utilisation de ce genre de forceur de type immédiatement après un `return`),

— de plagiat (recopie totale ou partielle d'un source tiers, dont IA. Toute expression de la forme `x` ou `!x` là où, de manière standard, est écrit en C, à l'inversion des objets de la comparaison près, `x != 0`, `x != '\0'`, `x != nullptr` ou `x == 0`, `x == '\0'`, `x == nullptr` sera considérée au moins comme suspecte),

— de méconnaissance du projet présenté lors de la soutenance,
— de méconnaissance du sujet du projet (ce support), y compris la FAQ.

Évaluation

La notation du projet est sensible à la propreté du code, à son homogénéité, aux performances temporelles et spatiales de l'exécutable, à la clarté des explications fournies dans le rapport et lors de la soutenance, à la maitrise complète du code, au passage sans encombre aux grils `valgrind` et `malloccp`.

MÀJ 21-05 Ajout du reste du corps de la section.

En dehors des cas rédhibitoires précédemment cités, la réalisation du développement et sa soutenance sont notées sur un maximum de 20 points répartis comme suit :

critère	*points*
à l'ordre 1 uniquement	4
aux ordres 1 et 2 uniquement	7
aux ordres 1 à 10 uniquement	10
jusqu'à des ordres supérieurs 10	12
option `-l`	1
option `-S`	1
rapport et soutenance	6

Lors de la soutenance, vous pouvez être interrogé·e sur des parties de votre propre code, amené·e à les justifier, les modifier ou les améliorer. Vous pouvez également être questionné·e sur votre utilisation d'interfaces de modules que vous n'avez pas développés.

Changements notables

Outre les corrections de coquilles et les reformulations :

30 mars 2026

Remplacement de `-w` par `-W`.
Ajout d'une quatrième option obligatoire : `-w` ou `--words`.
Remplacement de `--display-table` par `--table`.
Remplacement de `-?` par `-U`.
Ajouts de `MALLOCCP` dans le fichier `randtext/makefile`.
Précisions sur le contenu de l'archive et simplification dans la production de l'archive compressée.

21 mai 2026

Ajout de le section « Hors jeux ».
Compléments à la section « Évaluation ».

Foire aux questions

► 1 À propos de la fonction getopt dont vous avez parlé en cm dernièrement .
Je voulais savoir dans quels cas il est recommandé de l'utiliser pour gérer les arguments en ligne de commande, plutôt que de traiter directement argc et argv ?

15-03
Ajouts 1-4.

Mon petit Wiki me dit : « `getopt` est une fonction C POSIX utilisée pour analyser les options de la ligne de commande de type Unix/POSIX en C. Elle fait partie de la spécification POSIX et est commune à tous les systèmes de type Unix. »

Documentez-vous dessus. Vous pouvez l'utiliser, elle et ses dérivées pour les options longues. Attention toutefois à la mise en œuvre : « En particulier, les éventuels caractères, chaines et nombres magiques doivent être nommés. L'utilisation de variables globales doit être évitée. »

Vous pouvez tout aussi bien créer votre propre outil de traitement des paramètres qui figurent sur la ligne de commande.

Une recommandation ? Faites au mieux…

► 2 Pour les mots lus, vous dites que "seuls les préfixes d'une taille maximale fixée sont retenus". Ça ne serait pas plutôt "longueur" à la place de "taille" ?

Dans la suite du paragraphe dont vient l'extrait que vous citez, il est aussi fait état d'octets. « Taille » et « octets » sont utilisés pour simplifier le problème de lecture et de la mémorisation des mots. Contentez-vous de compter les octets : cela reviendra au même si les textes sources sont codés sur les caractères de base ; pour les autres textes sources, par exemple codés en UTF-8, tant pis pour les caractères qui seront coupés.

► 3 L'option `--width` s'applique aussi l'affichage de la table ?

Non, à l'affichage normal uniquement. De même pour `--limit`.

► 4 Pouvez-vous donner des exemples complets de tables ?

```
$ echo "a b a a b a b" | cat - > x.txt ; cat x.txt
a b a a b a b
$ ./randtext -t x.txt
──────⟩a
a─────⟩b─────⟩a─────⟩b─────⟩b
b─────⟩a─────⟩a─────⟩
$ ./randtext -t -o 2 x.txt
──────⟩──────⟩a
──────⟩a─────⟩b
a─────⟩b─────⟩a─────⟩a─────⟩
b─────⟩a─────⟩a─────⟩b
a─────⟩a─────⟩b
$ ./randtext -t -o 3 x.txt
──────⟩──────⟩──────⟩a
──────⟩──────⟩a─────⟩b
──────⟩a─────⟩b─────⟩a
a─────⟩b─────⟩a─────⟩a─────⟩b
b─────⟩a─────⟩a─────⟩b
a─────⟩a─────⟩b─────⟩a
b─────⟩a─────⟩b─────⟩
```

► 5 Est-ce que j'ai bien compris : l'implantation des n-uplets, c'est des tableaux de longueur n, et celle des listes de successeurs, c'est des listes dynamiques simplement chainées ? 31-03 Ajouts 5-7.

Oui, pourquoi pas, mais ce n'est pas la seule des implantations possibles… En reprenant vos suggestions, les *n*-uplets peuvent aussi être implantés par des listes, tout comme les listes de successeurs peuvent l'être par des tableaux dynamiques ! Les listes de successeurs implantées par des tableaux ont l'avantage d'un choix du successeur en temps constant.

► 6 J'ai besoin de modifier l'implantation des tables de hachage : il faut que j'associe à un n-uplet une liste. C'est donc pour ça qu'on à le droit de modifier le module `hashtable` ?

Euh… L'accès à vos listes se fait par un pointeur, non ? Que ce pointeur soit d'un type particulier, le vôtre en l'occurrence, c'est un pointeur compatible avec le type des valeurs des tables de hachage. Donc inutile de modifier `hashtable.c` pour ça ! Pour rappel, il vous est interdit de modifier quoique ce soit dans la partie en-tête `"hashtable.h"`.

► 7 Et si l'utilisateur donne plusieurs fois la même option sur la ligne de commande avec des valeurs différentes ou même identiques ? Doit-on traiter ce cas comme une erreur ? Sinon, laquelle des valeurs est la bonne ?

La convention usuelle est de ne pas râler et de ne considérer que la dernière valeur. Voici, dans l'abondante littérature sur les options, un extrait qui résume bien la philosophie générale :

> *Options begin with a dash and consist of a single character. GNU-style long options consist of two dashes and a keyword. The keyword can be abbreviated, as long as the abbreviation allows the option to be uniquely identified. If the option takes an argument, then the keyword is either immediately followed by an equals sign (=) and the argument's value, or the keyword and the argument's value are separated by whitespace. If a particular option with a value is given more than once, it is the last value that counts.*

► 8 Je me permets de vous contacter afin de savoir si un module implémentant les arbres AVL sera fourni pour le projet, ou si nous devons utiliser celui que nous avons développé lors du TP5. 09-04 Ajouts 8-12.

Fournir un module complet sur les AVL reviendrait à vous donner un corrigé (idéal ?) sur la fiche de TP numéro 5… Donc : recourrez à votre propre (partie) implantation.

► 9 La première clé du texte est dit que cela doit être n mot vide pour un ordre n. Comment doit-on représenter ce mot vide ? Est-ce avec `'\0'` ou d'une toute autre façon ?

Je crois qu'en C c'est `""`, du type `const char *`.

► 10 Lors d'une erreur sur la ligne de commande, il est écrit que l'on doit retourner 64 comme code d'erreur. Est-ce que l'ouverture d'un fichier, la lecture dans ce fichier ou sa fermeture compte comme une erreur sur la ligne de commande ? Si ce n'est pas le cas, doit-on retourner une erreur spécifique ou retourner `EXIT_FAILURE` "suffit" ?

Aucune des erreurs que vous mentionnez n'est une erreur de syntaxe ou un dépassement des limites imposées par l'exécutable aux arguments des options. Chacune des erreurs mentionnées est une erreur survenue lors de l'exécution. Donc, pour elles, `exit(EXIT_FAILURE)`, ou équivalent.

► 11 Pouvez-vous me préciser les contraintes attendues en termes de temps d'exécution et d'espace mémoire, notamment par rapport à l'exécutable fourni ?

Sur `lesmiserables.txt` (à trouver sur UniversiTICE), pour 100 mots : au plus 1 s à l'ordre 1 ; au plus 2 s à l'ordre 2 ; au plus 4 s à l'ordre 3.

▶ 12 Pourriez-vous m'indiquer comment utiliser le module malloccp pour tester les échecs d'allocation mémoire ?

Sur un petit texte comme `knock.txt` (à trouver sur UniversiTICE) et pour un ordre fixé, avec « `MALLOCCP = 0` » dans le fichier `makefile`, utilisez `valgrind` pour récupérer le nombre d'allocations effectuées, disons a. Ensuite, pour toutes les valeurs k de 1 à a : modifiez la première ligne du fichier `makefile` en « `MALLOCCP =` k » ; produisez l'exécutable ; testez l'exécutable sous `valgrind` sur `knock.txt`. Remettez « `MALLOCCP = 0` » comme première ligne du fichier `makefile`. Si une erreur est survenue lors de la boucle : remédiez au problème. Sinon : cela semble correct cet ordre ; essayez pour d'autres ordres.

▶ 13 J'aurais une question sur la syntaxe attendue pour la commande randtext. Dans l'énoncé, elle est indiquée sous la forme : randtext [OPTION...] [FILE] 10-04 Ajouts 13-13.

Je souhaitais savoir si l'ordre des arguments est strictement imposé, ou bien si une écriture comme : randtext [FILE] [OPTION...] serait également acceptée, ou considérée comme incorrecte (voire éliminatoire).

Par convention, les options figurent avant les opérandes. Consultez par exemple `https://pubs.opengroup.org/onlinepubs/9699919799/basedefs/V1_chap12.html#tag_12_02`.

▶ 14 Dans le sujet de du projet, il n'est pas mentionné la mise en forme à respecter concernant le compte rendu en pdf. Quelle mise en forme doit-on respecter ? 13-04 Ajouts 14-16.

La mise en forme est libre. L'insertion de captures d'écran (poids) est à prohiber.

▶ 15 Est-il utile d'utiliser la variable errno (et donc d'inclure errno.h) ?

Si vous utilisez les fonctions standard de transcodage, vous ne pouvez pas faire autrement pour contrôler la correction du transcodage.

▶ 16 Doit-on initialiser une locale grâce à la fonction setlocale (provenant de la bibliothèque locale.h) afin d'utiliser la fonction de comparaison strcoll plutôt que strcmp dans le cadre d'une comparaison de mots ?

Contrairement aux projets des années précédentes, aucun tri n'est attendu. Le recours à `strcoll` n'est donc pas nécessaire.

▶ 17 Est-ce que c'est acceptable d'avoir l'affichage de table de hachage à l'envers. 24-04 Ajouts 17-18.

Il n'y a aucune contrainte sur l'ordre d'affichage des clés ni sur l'ordre d'affichage des mots des listes des successeurs.

▶ 18 Je suis en train de finaliser la réflexion autour de la phase de génération de texte aléatoire, et je me pose une question technique concernant l'utilisation de la fonction rand().

Pour choisir un successeur de manière aléatoire, l'approche la plus classique consiste à utiliser un modulo par le nombre de successeurs disponibles. Cependant, je suppose qu'en procédant ainsi, la séquence de mots générée risque de différer de celle de votre exécutable de référence, même avec un germe (seed) identique. En effet, le résultat final dépend grandement de l'opération mathématique exacte choisie pour faire correspondre le tirage de rand() à un index du tableau.

Ma question concerne donc les modalités de l'évaluation automatique : vos scripts de correction s'attendent-ils à ce que notre sortie texte soit strictement identique à la vôtre pour un germe donné (ce qui nécessiterait d'utiliser exactement la même formule de tirage), ou bien vérifieront-ils la validité algorithmique des enchaînements produits vis-à-vis du texte source, ainsi que la bonne gestion de la mémoire ?

À partir d'un germe donné, votre exécutable doit toujours fournir le même résultat. L'exigence d'obtention des mêmes résultats que celui de mon exécutable obligerait à préciser les ordres de constitution, de consultation et de maintien des listes, ainsi que l'utilisation exacte de la fonction `rand` pour les consultations.

► 19 Je vous contacte car afin de vous demander si c'est possible d'expliquer l'option -L du projet. Je vous remercie par avance pour votre explication. 05-05 Ajouts 19-20.

Les options `-L` et `-W` ont été introduites pour améliorer l'esthétique des sorties normales.

Appelons « paragraphe » une suite de caractères de longueur maximale terminée par un caractère fin de ligne. Le texte source fourni comme premier exemple contient ainsi 14 paragraphes. Avec l'option `-L`, on cherche à rendre plus réaliste la sortie normale en mémorisant puis produisant les caractères fin de ligne. Voici ce que cela donne sur le texte source :

```
Knock: De quoi souffrez-vous?¶
Le Tambour: La. Ou peut-etre la... Entre les deux.¶
Knock: Juste entre les deux? ... Est-ce que ca vous chatouille, ou est-ce que↩
 ca ne vous grattouille pas davantage quand vous avez mange de la tete de↩
 veau a la vinaigrette?¶
Le Tambour: La. Ou peut-etre la... Entre les deux.¶
Knock: Juste entre les deux? ... Est-ce que ca ne serait pas plutot un rien a↩
 gauche, la, ou je mets mon doigt?¶
Le Tambour: Oui, on dirait que ca vous grattouille?¶
Le Tambour: La. Ou peut-etre la... Entre les deux.¶
Knock: Juste entre les deux? ... Est-ce que ca vous grattouille?¶
Le Tambour: Par ici.¶
Knock: Par ici... ou cela, par ici?¶
Le Tambour: Ca me grattouille. (Il medite.) Mais ca me grattouillerait plus.¶
```

En respectant le découpage originel, c'est plus « vivant ».

Si la composante « (nombre) de colonnes » de la « géométrie » de la fenêtre du terminal vaut c, l'utilisation de l'option `-W` c est toute indiquée : aucun mot n'apparaitra comme coupé par le bord droit de la fenêtre, sauf à ce que sa longueur soit strictement supérieure à c ou qu'elle soit égale à c et que le mot ne soit pas le premier d'un paragraphe. Le texte aléatoire fourni pour le tout premier exemple a été produit avec $c = 77$. Voici ce que cela donne avec $c = 80$, autrement dit le nombre de colonnes standard d'un terminal, sans plus (besoin de) montrer les continuations des lignes de texte mais en signifiant les espaces par « ␣ » :

```
Knock:␣De␣quoi␣souffrez-vous?␣Le␣Tambour:␣La.␣Ou␣peut-etre␣la...␣Entre␣les␣deux.
␣Knock:␣Juste␣entre␣les␣deux?␣...␣Est-ce␣que␣ca␣vous␣chatouille,␣ou␣est-ce␣que␣␣
␣ca␣ne␣vous␣grattouille␣pas␣davantage␣quand␣vous␣avez␣mange␣de␣la␣tete␣de␣veau␣a
␣la␣vinaigrette?␣Le␣Tambour:␣La.␣Ou␣peut-etre␣la...␣Entre␣les␣deux.␣Knock:␣Juste
␣entre␣les␣deux?␣...␣Est-ce␣que␣ca␣ne␣serait␣pas␣plutot␣un␣rien␣a␣gauche,␣la,␣ou
␣je␣mets␣mon␣doigt?␣Le␣Tambour:␣Oui,␣on␣dirait␣que␣ca␣vous␣grattouille?␣Le␣␣␣␣␣␣
␣Tambour:␣La.␣Ou␣peut-etre␣la...␣Entre␣les␣deux.␣Knock:␣Juste␣entre␣les␣deux?␣␣␣
␣␣...␣Est-ce␣que␣ca␣vous␣grattouille?␣Le␣Tambour:␣Par␣ici.␣Knock:␣Par␣ici...␣ou␣␣
␣cela,␣par␣ici?␣Le␣Tambour:␣Ca␣me␣grattouille.␣Knock,␣d'un␣air␣de␣profonde␣␣␣␣␣␣
␣concentration:␣Attention.␣Ne␣confondons␣pas.␣Est-ce␣que␣ca␣vous␣chatouille,␣ou␣
␣est-ce␣que␣ca␣me␣fait␣mal.␣Knock:␣Ah!␣ah!␣(Il␣medite␣d'un␣air␣de␣profonde␣␣␣␣␣␣
␣concentration:␣Attention.␣Ne␣confondons␣pas.
```

Pour finir, voici la production pour une combinaison des deux options, « `-L -W 80` » :

```
Knock:␣De␣quoi␣souffrez-vous?¶
Le␣Tambour:␣La.␣Ou␣peut-etre␣la...␣Entre␣les␣deux.¶
Knock:␣Juste␣entre␣les␣deux?␣...␣Est-ce␣que␣ca␣vous␣chatouille,␣ou␣est-ce␣que␣ca
␣ne␣vous␣grattouille␣pas␣davantage␣quand␣vous␣avez␣mange␣de␣la␣tete␣de␣veau␣a␣la
```

```
␣vinaigrette?¶
Le␣Tambour:␣La.␣Ou␣peut-etre␣la...␣Entre␣les␣deux.¶
Knock:␣Juste␣entre␣les␣deux?␣...␣Est-ce␣que␣ca␣ne␣serait␣pas␣plutot␣un␣rien␣a␣␣␣
␣gauche,␣la,␣ou␣je␣mets␣mon␣doigt?¶
Le␣Tambour:␣Oui,␣on␣dirait␣que␣ca␣vous␣grattouille?¶
Le␣Tambour:␣La.␣Ou␣peut-etre␣la...␣Entre␣les␣deux.¶
Knock:␣Juste␣entre␣les␣deux?␣...␣Est-ce␣que␣ca␣vous␣grattouille?¶
Le␣Tambour:␣Par␣ici.¶
Knock:␣Par␣ici...␣ou␣cela,␣par␣ici?¶
Le␣Tambour:␣Ca␣me␣grattouille.␣(Il␣medite.)␣Mais␣ca␣me␣grattouillerait␣plus.¶
```

► 20 Dans l'exécutable que vous avez mis en ligne, à quoi servent les options -r et -O ? Et que signifie MOUV qui apparait pour l'option -U ?

Les deux affichages « non normaux » permettent d'afficher la table et les mots. Ces deux affichages sont accessibles par `-t` et `-w`, ou `--table` et `--words`. Ils le sont également par l'option `-r` ou `--reveal` : `-r table` et `-r words`, ou `--reveal=table` et `--reveal=words`. Cette dernière option n'a été pensée qu'après avoir ajouté l'affichage des mots. Par souci de compatibilité ascendante, la plus ancienne `-t` et la nouvelle `-w` ont été maintenues.

L'option `-O` permet de ne faire que le point sur les options sélectionnées et, le cas échant, sur leurs valeurs courantes, sans donc lancer de lecture d'un texte, de construction de table et de génération aléatoire de texte. Avec `-O` pour seule option, la commande `randtext` affiche les valeurs par défaut des options. Pour l'exécutable fourni :

```
$ ./randtext -O
Current options: randtext -l 100 -o 1 -s 1 -S 63 -W 0
```

Voici ce qui est affiché par l'exécutable fourni dès que l'option `-U` figure sur la ligne de commande :

```
$ ./randtext -U
Usage: randtext [-Ltw?MOUV] [-l LIMIT] [-o ORDER] [-r FEATURE] [-s SEED]
    [-S SIZE] [-W WIDTH] [--limit=LIMIT] [--keep-end-of-lines] [--order=ORDER]
    [--reveal=FEATURE] [--seed=SEED] [--size=SIZE] [--table] [--words]
    [--width=WIDTH] [--help] [--man] [--options] [--usage] [--version] [-] [--]
    [FILE]
```

À l'instar de « `man --usage` », d'abord les options courtes sans paramètre, puis les courtes avec paramètre, puis les longues, puis les paramètres, optionnels ou obligatoires. Les (identificateurs des) options apparaissent dans le même ordre que celui pour les affichages de l'aide. Donc, pour les dernières sans paramètre, `-M` ou `--man`, `-O` ou `--options`, `-U` ou `--usage` puis `-V` ou `--version` ; d'où le « jeu de mots ». Notez en passant que les options courtes consécutives peuvent être groupées ; par exemple « `-L -t -o 2` » peut être raccourci en `-Lto2`.

► 21 J'ai une question concernant la génération aléatoire d'un mot à partir de sa clé. Si je prends une clé X et que cette clé à 3 valeurs noté u1 ; u2 ; u3 . Imaginons que l'aléatoire me retourne 2, le mot choisi est donc le mot u2. Est-ce que ce mot doit être supprimé de la structure dans lequel il est stocké, donc la clé reste inchangé (X) et les valeurs seraient dont u1 ; u3 , ou peut-on retomber dessus si jamais la clé X réapparait dans le texte, donc la clé et les valeurs resteraient inchangées ?

11-05 Ajouts 21-21.

Pour toute la durée de la phase de génération, aucun mot ne doit être retiré de la structure.

► 22 J'aurais quelques questions concernant le projet. Est-ce que le fait de ne pas développer toutes les options facultatives peut faire baisser la note du projet ? 14-05 Ajout 22-22.

Toutes : non. Mais permettre que `-o` puisse supporter des valeurs de 1 à 10 est un minimum.

► 23 Par rapport au traitement des options courtes, est-ce qu'il y a une différence entre écrire : 19-05 Ajout 23-23.

```
./randtext -W20
```

Ou

```
./randtext -W 20
```

Notamment pour l'espace entre l'option et la valeur qu'elle prend ? (Ici `-W` a été pris comme exemple mais cela s'applique sur toute les options courtes)

Non, aucune différence : les deux formes devraient pouvoir être supportés.

— *Quoi d'autre ?*